Python

Object Oriented Programming (OOP) di Python

Dalam paradigma OOP, sebuah code pemograman akan dibagi-bagi menjadi beberapa kelas-kelas. Kelas-Kelas tersebut kemudian akan dibungkus menjadi beberapa objek. Nah, objek-objek inilah yang kemudian akan saling berinteraksi dan membangun pemograman kita.

Objek adalah sesuatu yang memiliki identitas berupa atribut dan dapat memiliki behaviour (prilaku) berdasarkan suatu aksi.

Contoh: Misalkan kita sedang membangun sebuah objek yang dinamakan "MOBIL"

- Mobil memiliki atribut berupa jenis mobil (sedan, jeep, truk), merek, warna, setir, pedal, rem, ban, dll.
- Mobil memiliki prilaku atau aksi berupa berjalan cepat, berjalan rambat, mengerem, membunyikan klakson dll.

Mendefinisikan Class dalam OOP

Semua hal dalam python sebenarnya dapat kita pandang sebagai objek. Objek tersebut dibangun dari kelas. Jadi, kita dapat memandang bahwa kelas adalah suatu rancangan atau suatu *blueprint* untuk membangun suatu objek.

Untuk membuat class dalam python, dapat menggunakan keyword **class** diikuti dengan tanda petik dua (:)

In [1]:
```
# mendefinisikan kelas untuk membangun objek untuk mobil
class car :
    car_type = 'sedan'
    color = 'merah'


print(car.car_type)
print(car.color)
```

```
sedan
merah
```

Membuat Objek dari Suatu Kelas

Kita tahu bahwa kelas merupakan suatu konstruktor atau rancangan dari suatu objek. Untuk membuat suatu objek, kita dapat meng-assign kelas yang telah kita buat sebelumnya dalam suatu objek.

In [3]:
```
# Buat objek berupa “mobil_ayah” dari kelas mobil
mobil_ayah = car()

# print untuk melihat spesifikasi mobil_ayah
print("--- Informasi Mobil Ayah ---")
print("Jenis mobil ayah :",mobil_ayah.car_type)
print("Warna mobil ayah :",mobil_ayah.color)
```

```
--- Informasi Mobil Ayah ---
Jenis mobil ayah : sedan
Warna mobil ayah : merah
```

Dari output diatas bisa dilihat bahwa mobil ayah dan mobil adik merupakan dua objek yang berbeda atau tidak sama, walaupun berasal dari kelas yang sama

Fungsi `__init__`

Di bagian sebelumnya, kita sudah mendefinisikan class secara sederhana. Namun, hal itu belum cukup dan belum ada manfaatnya untuk aplikasi atau program yang kompleks. Untuk dapat memahami class lebih jauh, kita perlu mengetahui fungsi bawaan python yaitu `__init__`. Fungsi `__init__` digunakan untuk menginisiasi class yang akan membentuk objek. Biasanya fungsi ini berisikan nilai atau data yang akan mendeskripsikan properti suatu objek. Di fungsi ini juga dapat berisi operasi lain yang dilakukan ketika suatu objek diinisiasi.

Fungsi `__init___` didefinisikan seperti kita mendefinisikan suatu fungsi python.

Default Constructor

In [4]:
```
class Mobil:
    def __init__(self):
        self.jenis_mobil = "Sedan"
        self.warna_mobil = "Hitam"
        self.dapat_menyala = False

mobil_baru = Mobil()
print(mobil_baru.jenis_mobil)
```

```
Sedan
```

Di default Contructor, setiap objek **Mobil** yang dibuat akan memiliki jenis mobil "Sedan", warna "Hitam".

Parameterized Constructor

In [5]:
```
class Mobil:
    def __init__(self, jenis, warna, ada_aki):
        self.jenis_mobil = jenis
        self.warna_mobil = warna
        self.dapat_menyala = self.starter(ada_aki)

    def starter(self, aki):
        if aki:
            return True
        else:
            return False

# Membuat objek mobil_ayah dan mobil_adik dari konstruktor kelas
mobil_ayah = Mobil("sedan", "merah", True)
mobil_adik = Mobil("jeep", "hijau", False)
```

In [6]:
```
print(f"""
--- Informasi Mobil Ayah ---
Jenis mobil: {mobil_ayah.jenis_mobil}
Warna mobil: {mobil_ayah.warna_mobil}
Apakah menyala? {mobil_ayah.dapat_menyala}
"""
)

print(f"""
--- Informasi Mobil Adik ---
Jenis mobil: {mobil_adik.jenis_mobil}
Warna mobil: {mobil_adik.warna_mobil}
Apakah menyala? {mobil_adik.dapat_menyala}
""")
```

```
--- Informasi Mobil Ayah ---
Jenis mobil: sedan
Warna mobil: merah
Apakah menyala? True


--- Informasi Mobil Adik ---
Jenis mobil: jeep
Warna mobil: hijau
Apakah menyala? False
```

Hands On! Buat suatu kelas bernama User

In [9]:
```
# Membuat kelas User
class User:
    def __init__(self, username, email, umur):
        self.username = username
        self.email = email
        self.umur = umur



    def info_user(self):
```

```
        return f"Username: {self.username}, Email: {self.email}, Umur: {self.umur}"

# Membuat objek dari kelas User
user1 = User("Hafiz", "hafiz@email.com", 23)

# Menampilkan informasi pengguna
print(user1.info_user())
```

```
Username: Hafiz, Email: hafiz@email.com, Umur: 23
```

Instance Variable

Selain memiliki atribut, sebuah objek juga dapat memiliki method (atau biasa kita juga mengenalnya sebagai fungsi). Method ini akan dilaksanakan ketika kita memanggil objek kita beserta method yang akan dilakukan.

In [10]:

```
# Membuat class Mobil

# Mobil tersebut akan menghabiskan 5 liter bensin setiap kilometernya

class Mobil:
    def __init__(self,jenis,warna,bensin = 100) :
        self.jenis_mobil = jenis
        self.warna_mobil = warna
        self.liter_bensin = bensin
        self.kilometer = 0

    def jalan_maju(self,jumlah_km) :
      # aksi apabila ada bensin (bensin tidak kurang dari sama dengan nol)
      if self.liter_bensin > 0 :
        # menghitung bensin yang masih tersisa untuk melakukan aksi
        sisa_bensin = self.liter_bensin - jumlah_km*5

        # apabila sisa bensin lebih dari nol
        if sisa_bensin > 0 :
            self.kilometer += jumlah_km
            self.liter_bensin = sisa_bensin
            print(f"mobil maju {jumlah_km} km, sekarang mobil berada di posisi {sel
        # apabila sisa bensin kurang dari nol
        else :
            self.kilometer += self.liter_bensin/5
            print(f"Bensin tidak cukup! Mobil hanya maju {self.liter_bensin/5} km.
            self.liter_bensin -= jumlah_km*5

      # apabila bensin tidak ada (minus atau sama dengan nol, maka mobil tidak bisa
      else :
            print(f"mobil tidak ada bensin, jadi tidak bisa maju!")

    def jalan_mundur(self,jumlah_km) :
      # aksi apabila ada bensin (bensin tidak kurang dari sama dengan nol)
        if self.liter_bensin > 0 :
            # menghitung bensin yang masih tersisa untuk melakukan aksi
                sisa_bensin = self.liter_bensin - jumlah_km*5

                # apabila sisa bensin lebih dari nol
```

```
                if sisa_bensin > 0 :
                    self.kilometer -= jumlah_km
                    self.liter_bensin = sisa_bensin
                    print(f"mobil maju {jumlah_km} km, sekarang mobil berada di pos
                # apabila sisa bensin kurang dari nol
                else :
                    self.kilometer -= self.liter_bensin/5
                    print(f"Bensin tidak cukup! Mobil hanya mundur {self.liter_bens
                    self.liter_bensin -= jumlah_km*5

            # apabila bensin tidak ada (minus atau sama dengan nol,
            # maka mobil tidak bisa melakukan aksi apa-apa)
        else :
            print(f"mobil tidak ada bensin, jadi tidak bisa maju!")

    def isi_bensin(self):
        self.liter_bensin = 100
        print("bensin mobil sudah terisi penuh!")
```

In [11]:
```
mobil_ayah = Mobil("jeep", "merah")
mobil_ayah.jalan_maju(10)
mobil_ayah.jalan_mundur(30)
mobil_ayah.jalan_maju(5)
mobil_ayah.isi_bensin()
mobil_ayah.jalan_maju(2)
```

```
mobil maju 10 km, sekarang mobil berada di posisi 10 km dari titik awal. Bensin ters
isa tinggal 50
Bensin tidak cukup! Mobil hanya mundur 10.0 km. Sekarang mobil di posisi 0.0 dari po
sisi awal. Bensin sudah habis!
mobil tidak ada bensin, jadi tidak bisa maju!
bensin mobil sudah terisi penuh!
mobil maju 2 km, sekarang mobil berada di posisi 2.0 km dari titik awal. Bensin ters
isa tinggal 90
```

Thank you :)

Hands On

Buatlah class Lingkaran yang memiliki atribut : jari-jari 10 cm, pi = 3.14159 dengan method Luas Lingkaran dan Keliling Lingkaran

In [12]:
```
class Lingkaran:
    def __init__(self):
        self.jari_jari = 10
        self.pi = 3.14159

    def luas_lingkaran(self):
        return self.pi * self.jari_jari ** 2

    def keliling_lingkaran(self):
        return 2 * self.pi * self.jari_jari
```

```
lingkaran = Lingkaran()


print("Luas Lingkaran:", lingkaran.luas_lingkaran(), "cm2")
print("Keliling Lingkaran:", lingkaran.keliling_lingkaran(), "cm")
```

```
Luas Lingkaran: 314.159 cm2
Keliling Lingkaran: 62.8318 cm
```